Free Choice 1 Analysis

March 7, 2026

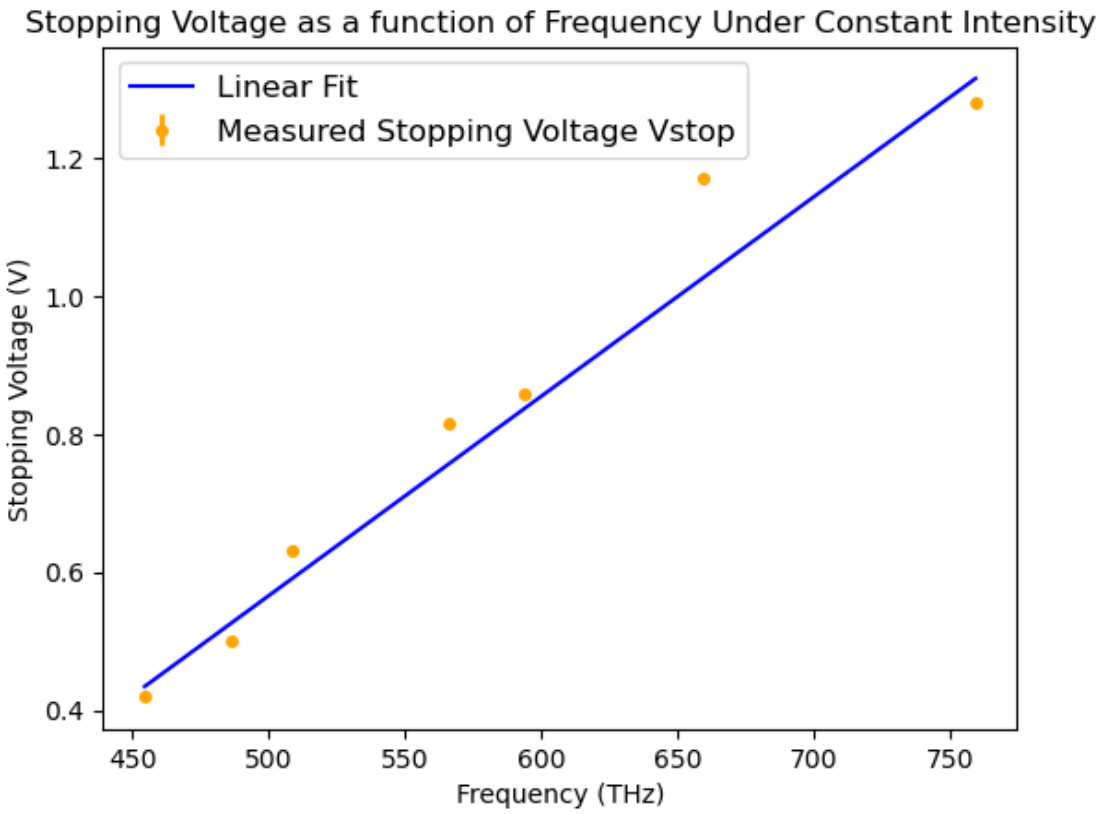
[3]:
```
#import all the necessary libraries
import numpy as np
from scipy.optimize import curve_fit as cf
import matplotlib.pyplot as plt
plus_or_minus= '\u00B1'

#read the lab data for exercise 1
data_ex1 = np.loadtxt("Free Choice 1 Data - Exercise 1.
 ↪csv",delimiter=',',comments="#",unpack=True, skiprows=1)
V_ex1 = data_ex1[13]
uV_ex1 = data_ex1[14]
f_ex1 = data_ex1[2]
uf_ex1 = data_ex1[3]

e = 1.602*10**(-19)
#construct the model that relates stopping voltage to frequency of LED
def photoelectric_effect(f, h, E0):
    V = (h*f/e) - (E0/e)
    return V

#curvefit time
popt_ex1, pcov_ex1 = cf(photoelectric_effect, f_ex1, V_ex1,␣
 ↪absolute_sigma=True, sigma = uV_ex1)
f0_predicted = (popt_ex1[1])/(popt_ex1[0])
uf0_predicted = (f0_predicted)*np.sqrt(((np.sqrt(pcov_ex1[1,1])/
 ↪popt_ex1[1])**2)+((np.sqrt(pcov_ex1[0,0])/popt_ex1[0])**2))

print(f"predicted h is {popt_ex1[0]:.5g}{plus_or_minus}{np.sqrt(pcov_ex1[0,0]):.
 ↪1g}")
print(f"predicted E0 is {popt_ex1[1]:.5g}{plus_or_minus}{np.sqrt(pcov_ex1[1,1]):
 ↪.1g}")
print(f"predicted f0 is {f0_predicted:.5g}{plus_or_minus}{uf0_predicted:.1g}")

#Chi-square Calculation
def chi2(measured,predicted,error):
    return np.sum( (measured - predicted)**2 / error**2 )
```

```
def chi2reduced(y_measure, y_predict, errors, number_of_parameters):
    return chi2(y_measure, y_predict, errors)/(y_measure.size -␣
 ↪number_of_parameters)

chi2Value = chi2(V_ex1, photoelectric_effect(f_ex1, popt_ex1[0],␣
 ↪popt_ex1[1]),uV_ex1)
RChi2Value = chi2reduced(V_ex1, photoelectric_effect(f_ex1, popt_ex1[0],␣
 ↪popt_ex1[1]),uV_ex1,3)
print(f"chi2 is {chi2Value:.5g}")
print(f"reduced chi2 is {RChi2Value:.5g}")
```

```
predicted h is 4.6232e-34±5e-37
predicted E0 is 1.4052e-19±3e-22
predicted f0 is 3.0395e+14±7e+11
chi2 is 28804
reduced chi2 is 7200.9
```

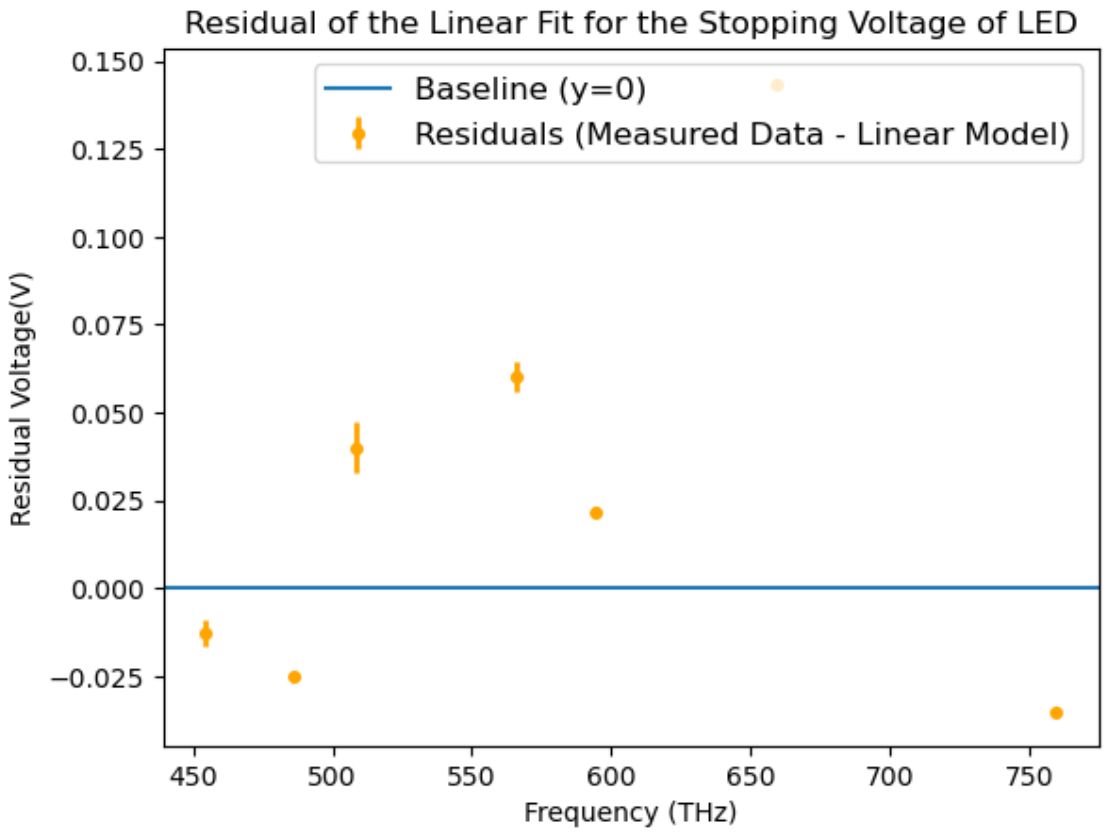
[8]:
```
plt.figure()
plt.title("Stopping Voltage as a function of Frequency Under Constant␣
 ↪Intensity")
plt.xlabel("Frequency (THz)")
plt.ylabel("Stopping Voltage (V)")
plt.errorbar(f_ex1*(10**-12), V_ex1,␣
 ↪yerr=uV_ex1,color='orange',marker='o',ls='',lw=2,ms=4, label ="Measured␣
 ↪Stopping Voltage Vstop")
plt.plot(f_ex1*(10**-12), photoelectric_effect(f_ex1, popt_ex1[0],␣
 ↪popt_ex1[1]), "b-", label = "Linear Fit")
plt.legend(fontsize = 12)
```

[8]:
```
<matplotlib.legend.Legend at 0x24295642490>
```



[9]:
```
#PLOT the resisdual plot
plt.errorbar(f_ex1*(10**-12), V_ex1-photoelectric_effect(f_ex1, popt_ex1[0],␣
 ↪popt_ex1[1]),␣
 ↪yerr=uV_ex1,color='orange',marker='o',ls='',lw=2,ms=4,label="Residuals␣
 ↪(Measured Data - Linear Model)")
plt.axhline(y=0, label = "Baseline (y=0)")
plt.xlabel("Frequency (THz)")
plt.ylabel("Residual Voltage(V)")
plt.title("Residual of the Linear Fit for the Stopping Voltage of LED")
plt.legend(fontsize = 12)
```

[9]:
```
<matplotlib.legend.Legend at 0x24295c89d10>
```

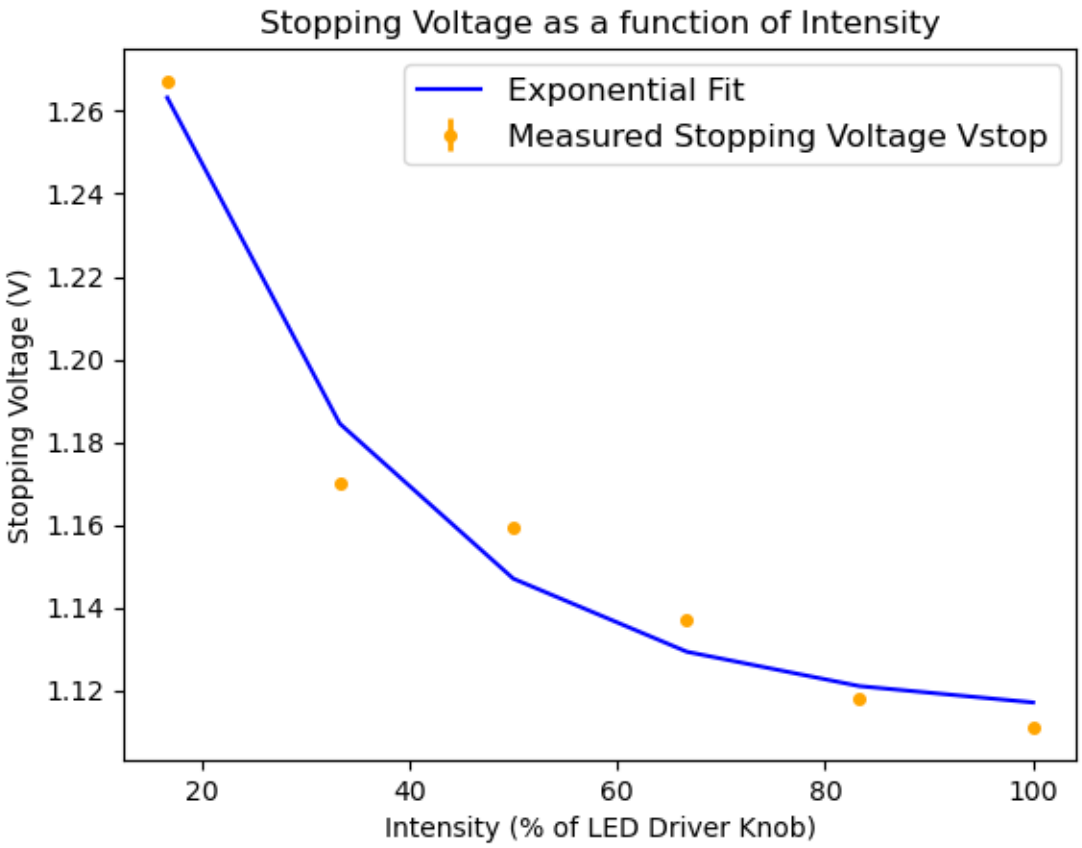


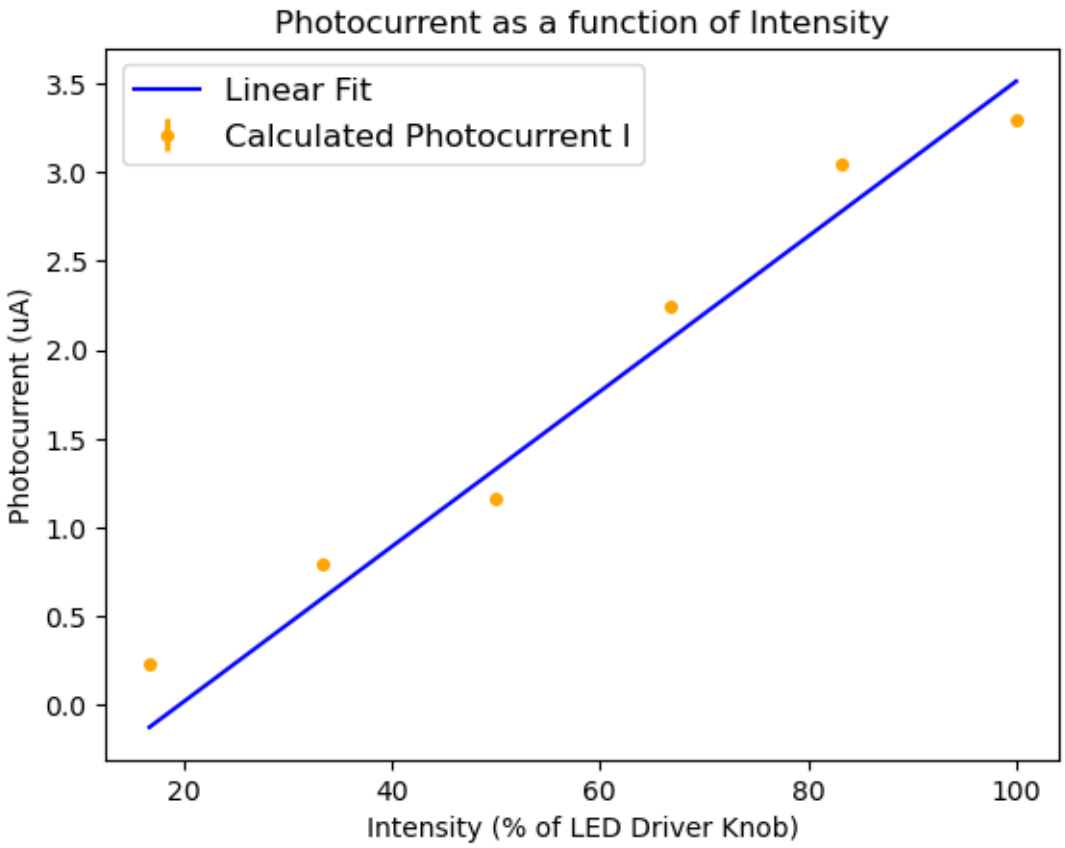
[14]:
```
#read the lab data for exercise 2
data_ex2 = np.loadtxt("Free Choice 1 Data - Exercise 2.
 ↪csv",delimiter=',',comments="#",unpack=True, skiprows=1)
intensity_ex2 = data_ex2[0]
uintensity_ex2 = 2
I_ex2 = data_ex2[3]
uI_ex2 = data_ex2[4]
V_ex2 = data_ex2[5]
uV_ex2 = data_ex2[6]

def linear_model(x, m, b):
    return ((m*x)+b)
def exponential_model(x, a, b, c):
    return ((a*(b**x))+c)

popt_ex2_I, pcov_ex2_I = cf(linear_model, intensity_ex2, I_ex2,␣
 ↪absolute_sigma=True, sigma = uI_ex2)
print(f"predicted m is {popt_ex2_I[0]:.5g}{plus_or_minus}{np.
 ↪sqrt(pcov_ex2_I[0,0]):.1g}")
```

```
print(f"predicted b is {popt_ex2_I[1]:.5g}{plus_or_minus}{np.
 ↪sqrt(pcov_ex2_I[1,1]):.1g}")

popt_ex2_V, pcov_ex2_V = cf(exponential_model, intensity_ex2, V_ex2,␣
 ↪absolute_sigma=True, sigma = uV_ex2)
print(f"predicted a is {popt_ex2_V[0]:.5g}{plus_or_minus}{np.
 ↪sqrt(pcov_ex2_V[0,0]):.1g}")
print(f"predicted b is {popt_ex2_V[1]:.5g}{plus_or_minus}{np.
 ↪sqrt(pcov_ex2_V[1,1]):.1g}")
print(f"predicted c is {popt_ex2_V[2]:.5g}{plus_or_minus}{np.
 ↪sqrt(pcov_ex2_V[2,2]):.1g}")


plt.figure(1)
plt.title("Stopping Voltage as a function of Intensity")
plt.xlabel("Intensity (% of LED Driver Knob)")
plt.ylabel("Stopping Voltage (V)")
plt.errorbar(intensity_ex2, V_ex2,␣
 ↪yerr=uV_ex2,color='orange',marker='o',ls='',lw=2,ms=4, label ="Measured␣
 ↪Stopping Voltage Vstop")
plt.plot(intensity_ex2, exponential_model(intensity_ex2, popt_ex2_V[0],␣
 ↪popt_ex2_V[1], popt_ex2_V[2]), "b-", label = "Exponential Fit")
plt.legend(fontsize = 12)

plt.figure(2)
plt.title("Photocurrent as a function of Intensity")
plt.xlabel("Intensity (% of LED Driver Knob)")
plt.ylabel("Photocurrent (uA)")
plt.errorbar(intensity_ex2, I_ex2*(10**6),␣
 ↪yerr=uI_ex2*(10**6),color='orange',marker='o',ls='',lw=2,ms=4, label␣
 ↪="Calculated Photocurrent I")
plt.plot(intensity_ex2, linear_model(intensity_ex2, popt_ex2_I[0],␣
 ↪popt_ex2_I[1])*(10**6), "b-", label = "Linear Fit")
plt.legend(fontsize = 12)
```

```
predicted m is 4.3639e-08±3e-12
predicted b is -8.5267e-07±2e-10
predicted a is 0.31766±0.0002
predicted b is 0.95598±3e-05
predicted c is 1.1135±3e-05
```

[14]: <matplotlib.legend.Legend at 0x24296078550>


Stopping Voltage as a function of Intensity
Exponential Fit
Measured Stopping Voltage Vstop
Stopping Voltage (V)
1.26
1.24
1.22
1.20
1.18
1.16
1.14
1.12
20
40
60
80
100
Intensity (% of LED Driver Knob)


Photocurrent as a function of Intensity
Linear Fit
Calculated Photocurrent I
Photocurrent (uA)
Intensity (% of LED Driver Knob)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
20
40
60
80
100

[ ]: